

CIP-B105 — COMPUTER FORENSICS CASE STUDY II
Case Study 4 — Morris Worm Investigation

Submitted by
CHUKWUEBUKA BASIL OKORIE
Student ID: C11/26/DFIT/17302

Cohort 11 — Digital Forensic Internship Training
International Cybersecurity and Digital Forensics Academy (ICDFA)
Nasarawa, Nigeria

Instructor:
Mr. Aminu Idris, AMCPN

Executive Summary

This report presents a process, port and file-timestamp investigation carried out on the SEED lab VM following the Morris Worm training exercise documented in the earlier Controlled Attack Observation report. The work was performed from a single shell on the SEED VM (seed@VM), inside the working directory /home/seed/worm/Labsetup/worm, and covers process listing, process-tree inspection, listening-port enumeration, file hashing and file-timestamp analysis.

Process evidence was captured with ps in several forms (Figures 1–4, 6–8). The most significant finding is in the full-format listing at Figure 7, which shows process 20220 running “/bin/sh -c ping -q -i2 1.2.3.4” with parent process ID 2376, and its child process 20221 running “ping -q -i2 1.2.3.4”. Because PID 2376 is not the attacker’s own login shell (PID 2967), this pairing is consistent with the ping command being spawned by the injected shellcode reviewed in the earlier report rather than typed directly by the user.

File integrity and timing were captured with md5sum and stat (Figures 10–13). worm.py hashes to 972737b166472d1a97085ea56c5f5159 (MD5) and badfile hashes to eb73555f9e837388b1f6380bb6313cca (MD5). stat shows worm.py’s content unchanged since its original authoring on 2026-02-10, while it was accessed (read/executed) on 2026-09-24 at 06:40:38 EDT. badfile’s access, modify and change timestamps are all identical at 2026-09-24 06:57:44 EDT, indicating the file was freshly written at that single moment — consistent with worm.py generating a new badfile as part of this run. The host’s local time zone was confirmed as EDT (UTC-4) using the date command (Figure 11).

Process Analysis

```
[09/24/26]seed@VM:~$ ps
  PID TTY          TIME CMD
 2967 pts/0    00:00:00 bash
 20220 pts/0    00:00:00 sh
 20221 pts/0    00:00:00 ping
 20422 pts/0    00:00:00 ps
[09/24/26]seed@VM:~$
```

Figure 1. ps in the current shell, listing four processes: bash (2967), sh (20220), ping (20221) and ps itself (20422).

```
[09/24/26]seed@VM:~$ ps --help

Usage:
ps [options]

Try 'ps --help <simple|list|output|threads|misc|all>'
or 'ps --help <s|l|o|t|m|a>'
for additional help text.

For more details see ps(1).
[09/24/26]seed@VM:~$ ps --help simple

Usage:
ps [options]

Basic options:
-A, -e          all processes
-a             all with tty, except session leaders
-a            all with tty, including other users
-d             all except session leaders
-N, --deselect  negate selection
-r             only running processes
-T             all processes on this terminal
-x            processes without controlling ttys

For more details see ps(1).
[09/24/26]seed@VM:~$
```

Figure 2. ps --help and ps --help simple output, documenting the option syntax used elsewhere in this investigation.

```
[09/24/26] seed@VM:~$ ps -e | head
  PID TTY          TIME CMD
    1 ?           00:00:13 systemd
    2 ?           00:00:00 kthreadd
    3 ?           00:00:00 rcu_gp
    4 ?           00:00:00 rcu_par_gp
    6 ?           00:00:00 kworker/0:0H-kblockd
    9 ?           00:00:00 mm_percpu_wq
   10 ?           00:00:03 ksoftirqd/0
   11 ?           00:00:02 rcu_sched
   12 ?           00:00:00 migration/0
[09/24/26] seed@VM:~$ █
```

Figure 3. `ps -e | head`, showing the start of the full system-wide process list beginning with `systemd` and the kernel worker threads.

```
[09/24/26] seed@VM:~$ ps
  PID TTY          TIME CMD
 2967 pts/0        00:00:00 bash
20220 pts/0        00:00:00 sh
20221 pts/0        00:00:01 ping
20642 pts/0        00:00:00 ps
[09/24/26] seed@VM:~$ firefox
[09/24/26] seed@VM:~$ █
```

Figure 4. Repeat of `ps` in the current shell (bash 2967, sh 20220, ping 20221, ps 20642), followed by launching `firefox`.

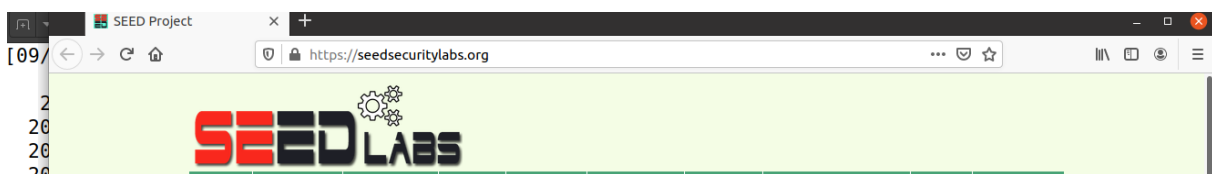


Figure 5. Firefox opened to seedsecuritylabs.org, confirming the firefox process started in Figure 4.

```
[09/24/26] seed@VM:~$ ps -eH --forest
PID TTY          TIME CMD
  2 ?           00:00:00 kthreadd
  3 ?           00:00:00 \_ rcu_gp
  4 ?           00:00:00 \_ rcu_par_gp
  6 ?           00:00:00 \_ kworker/0:0H-kblockd
  9 ?           00:00:00 \_ mm_percpu_wq
 10 ?           00:00:04 \_ ksoftirqd/0
 11 ?           00:00:02 \_ rcu_sched
 12 ?           00:00:00 \_ migration/0
 13 ?           00:00:00 \_ idle_inject/0
 14 ?           00:00:00 \_ cpuhp/0
 15 ?           00:00:00 \_ kdevtmpfs
 16 ?           00:00:00 \_ netns
 17 ?           00:00:00 \_ rcu_tasks_kthre
 18 ?           00:00:00 \_ kauditd
 19 ?           00:00:00 \_ khungtaskd
 20 ?           00:00:00 \_ oom_reaper
 21 ?           00:00:00 \_ writeback
 22 ?           00:00:00 \_ kcompactd0
 23 ?           00:00:00 \_ ksm
 24 ?           00:00:00 \_ khugepaged
 70 ?           00:00:00 \_ kintegrityd
 71 ?           00:00:00 \_ kblockd
 72 ?           00:00:00 \_ blkcg_punt_bio
 73 ?           00:00:00 \_ tpm_dev_wq
 74 ?           00:00:00 \_ ata_sff
 75 ?           00:00:00 \_ md
 76 ?           00:00:00 \_ edac-poller
 77 ?           00:00:00 \_ devfreq_wq
 78 ?           00:00:00 \_ watchdogd
 82 ?           00:00:03 \_ kswapd0
 83 ?           00:00:00 \_ kswapd0
```

Figure 6. ps -eH --forest (process-tree view). The visible portion of the capture shows the kernel-thread hierarchy rooted under kthreadd.

```
[09/24/26]seed@VM:~$ ps -f
UID      PID     PPID  C  STIME TTY          TIME CMD
seed     2967     2959  0  03:03 pts/0      00:00:00 bash
seed     20220    2376  0  07:00 pts/0      00:00:00 /bin/sh -c ping -q -i2 1.2.3.4
seed     20221    20220  0  07:00 pts/0      00:00:01 ping -q -i2 1.2.3.4
seed     21020    2967  0  07:17 pts/0      00:00:00 ps -f
[09/24/26]seed@VM:~$
```

Figure 7. ps -f (full-format listing) showing process 20220, “/bin/sh -c ping -q -i2 1.2.3.4”, with parent process ID 2376, and its child process 20221, “ping -q -i2 1.2.3.4” — distinct from the attacker’s own login shell, PID 2967.

```
[09/24/26]seed@VM:~$ ps -aux
USER      PID  %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.3 102464  7728 ?        Ss   03:02   0:14 /sbin/init splash
root         2   0.0  0.0      0     0 ?        S    03:02   0:00 [kthreadd]
root         3   0.0  0.0      0     0 ?        I<   03:02   0:00 [rcu_gp]
root         4   0.0  0.0      0     0 ?        I<   03:02   0:00 [rcu_par_gp]
root         6   0.0  0.0      0     0 ?        I<   03:02   0:00 [kworker/0:0H-kblockd]
root         9   0.0  0.0      0     0 ?        I<   03:02   0:00 [mm_percpu_wq]
root        10   0.0  0.0      0     0 ?        S    03:02   0:04 [ksoftirqd/0]
root        11   0.0  0.0      0     0 ?        I    03:02   0:02 [rcu_sched]
root        12   0.0  0.0      0     0 ?        S    03:02   0:00 [migration/0]
root        13   0.0  0.0      0     0 ?        S    03:02   0:00 [idle_inject/0]
root        14   0.0  0.0      0     0 ?        S    03:02   0:00 [cpuhp/0]
root        15   0.0  0.0      0     0 ?        S    03:02   0:00 [kdevtmpfs]
root        16   0.0  0.0      0     0 ?        I<   03:02   0:00 [netns]
root        17   0.0  0.0      0     0 ?        S    03:02   0:00 [rcu_tasks_kthre]
root        18   0.0  0.0      0     0 ?        S    03:02   0:00 [kauditd]
root        19   0.0  0.0      0     0 ?        S    03:02   0:00 [khungtaskd]
root        20   0.0  0.0      0     0 ?        S    03:02   0:00 [oom_reaper]
root        21   0.0  0.0      0     0 ?        I<   03:02   0:00 [writeback]
root        22   0.0  0.0      0     0 ?        S    03:02   0:00 [kcompactd0]
root        23   0.0  0.0      0     0 ?        SN   03:02   0:00 [ksmd]
root        24   0.0  0.0      0     0 ?        SN   03:02   0:00 [khugepaged]
root        70   0.0  0.0      0     0 ?        I<   03:02   0:00 [kintegrityd]
root        71   0.0  0.0      0     0 ?        I<   03:02   0:00 [kblockd]
root        72   0.0  0.0      0     0 ?        I<   03:02   0:00 [blkcg_punt_bio]
root        73   0.0  0.0      0     0 ?        I<   03:02   0:00 [tpm_dev_wq]
root        74   0.0  0.0      0     0 ?        I<   03:02   0:00 [ata_sff]
root        75   0.0  0.0      0     0 ?        I<   03:02   0:00 [md]
root        76   0.0  0.0      0     0 ?        I<   03:02   0:00 [edac-poller]
root        77   0.0  0.0      0     0 ?        I<   03:02   0:00 [devfreq_wq]
root        78   0.0  0.0      0     0 ?        S    03:02   0:00 [watchdogd]
root        82   0.0  0.0      0     0 ?        S    03:02   0:00 [ksysmon]

```

Figure 8. ps -aux, a full listing of system processes with owning user, CPU/memory usage and state for each entry.

Network Port Analysis

```
[09/24/26]seed@VM:~$ netstat -tulpn
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:44467        0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23            0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631         0.0.0.0:*               LISTEN      -
tcp6       0      0 :::21                  :::*                     LISTEN      -
tcp6       0      0 :::22                  :::*                     LISTEN      -
tcp6       0      0 :::1:631               :::*                     LISTEN      -
udp        0      0 0.0.0.0:5353          0.0.0.0:*               -          -
udp        0      0 0.0.0.0:631           0.0.0.0:*               -          -
udp        0      0 0.0.0.0:48073         0.0.0.0:*               -          -
udp        0      0 127.0.0.53:53         0.0.0.0:*               -          -
udp6       0      0 :::5353                :::*                     -          -
udp6       0      0 :::46800               :::*                     -          -
[09/24/26]seed@VM:~$
```

Figure 9. netstat -tulpn, listing active TCP/UDP listening ports on this host at the time of capture: 21, 22, 23, 53, 631 and associated UDP services.

File Hash and Timestamp Analysis

```
[09/24/26]seed@VM:~/.../worm$ pwd
/home/seed/worm/Labsetup/worm
[09/24/26]seed@VM:~/.../worm$ ll
total 8
-rw-rw-r-- 1 seed seed 500 Sep 24 06:57 badfile
-rwxrwxr-x 1 seed seed 2745 Feb 10 2026 worm.py
[09/24/26]seed@VM:~/.../worm$ md5sum badfile
eb73555f9e837388b1f6380bb6313cca badfile
[09/24/26]seed@VM:~/.../worm$ md5sum worm.py
972737b166472d1a97085ea56c5f5159 worm.py
[09/24/26]seed@VM:~/.../worm$
```

Figure 10. In /home/seed/worm/Labsetup/worm: directory listing showing badfile (500 bytes, dated Sep 24) and worm.py (2745 bytes, dated Feb 10 2026), followed by their MD5 hashes — badfile: eb73555f9e837388b1f6380bb6313cca; worm.py: 972737b166472d1a97085ea56c5f5159.

```
[09/24/26]seed@VM:~/.../worm$ date
Thu 24 Sep 2026 07:29:07 AM EDT
[09/24/26]seed@VM:~/.../worm$
```

Figure 11. Output of date, confirming the host's local time zone as EDT (UTC-4) at the time of this investigation.

```
[09/24/26]seed@VM:~/.../worm$ stat worm.py
File: worm.py
Size: 2745          Blocks: 8          IO Block: 4096   regular file
Device: 805h/2053d Inode: 1573901   Links: 1
Access: (0775/-rwxrwxr-x)  Uid: ( 1000/   seed)   Gid: ( 1000/   seed)
Access: 2026-09-24 06:40:38 027203644 -0400
Modify: 2026-02-10 14:51:34 313621165 -0500
Change: 2026-02-10 15:03:40 174226783 -0500
Birth: -
[09/24/26]seed@VM:~/.../worm$ █
```

Figure 12. stat worm.py, showing Access 2026-09-24 06:40:38 (-0400), Modify 2026-02-10 14:51:34 (-0500) and Change 2026-02-10 15:03:40 (-0500).

```
[09/24/26]seed@VM:~/.../worm$ stat badfile
File: badfile
Size: 500          Blocks: 8          IO Block: 4096   regular file
Device: 805h/2053d Inode: 1573900   Links: 1
Access: (0664/-rw-rw-r--) Uid: ( 1000/   seed)   Gid: ( 1000/   seed)
Access: 2026-09-24 06:57:44.245249837 -0400
Modify: 2026-09-24 06:57:44.161165148 -0400
Change: 2026-09-24 06:57:44.161165148 -0400
Birth: -
[09/24/26]seed@VM:~/.../worm$ █
```

Figure 13. stat badfile, showing Access, Modify and Change all recorded at 2026-09-24 06:57:44 (-0400).

Conclusion

The process evidence in this investigation corroborates, rather than merely repeats, the attack-launch findings of the earlier Controlled Attack Observation report. Figure 7 shows a ping process running under a shell whose parent process ID, 2376, does not belong to the analyst's own login session. Read alongside the `worm.py` source reviewed previously — whose shellcode issues exactly this kind of ping as a marker of successful execution — the most reasonable reading of this process pair is that it was spawned by injected code rather than typed at a prompt. This is process-level evidence of code execution consistent with a successful buffer-overflow exploit, independent of the console banners captured in the earlier report.

The file evidence is similarly consistent. `worm.py`'s modify and change timestamps remain fixed at 2026-02-10, meaning its content has not been altered since it was originally written, while its access time of 2026-09-24 06:40:38 records that it was read or executed during this exercise. `badfile`'s access, modify and change timestamps are identical at 2026-09-24 06:57:44, which is the signature of a file being written once and not subsequently touched — consistent with `worm.py` generating a fresh `badfile` as its first action at run time, matching the injection step described in the lab design.